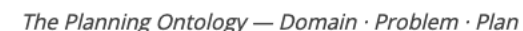


Planning Ontology

Explainable AI



maPO PURL purl.org/ai4s/ontology/planning/multi-agent



Tutorial · Mon, June 29 · 09:00–12:30

What This Tutorial Covers

*One idea, end to end: **structure planning knowledge as an ontology**, then exploit it for performance and explanation — from single-agent PDDL to multi-agent path finding.*

1

Background & Motivation

Automated planning, why PDDL is structured, and the case for an ontology.

2

Single-Agent: The Planning Ontology

Scope, competency questions, and two use cases — planner selection & explanation.

3

The Planning Ontology Tool

PDDL → knowledge graph → SPARQL, live inside Planning.Domains.

4

Multi-Agent: maPO

Explainable MAPF — schema, provenance, the OMEGA dashboard & evaluation.

PART 1

Background & Motivation

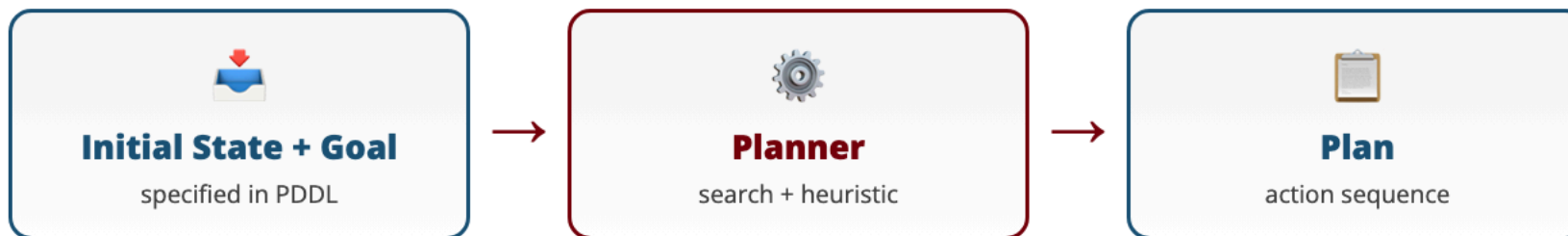
Automated planning & structured knowledge

PDDL is structured

Why an ontology?

Automated Planning, at Scale

Automated planning finds an ordered sequence of actions that transforms an **initial state** into a **goal state**.



A vast ecosystem

Many domains, planners, search algorithms & heuristics — each combination differs in quality, cost & optimality.

Rich empirical data

The International Planning Competition (IPC) yields performance results across many domains.

But unstructured

This metadata is rarely captured in a structured, reasoning-friendly form — the gap we address.

Planning is Structured: PDDL

blocksworld · domain & problem (PDDL)

```
(define (domain blocksworld)
  (:predicates (on ?x ?y) (clear ?x)
               (ontable ?x) (holding ?x)
               (handempty))
  (:action stack :parameters (?x ?y)
   :precondition (and (clear ?y) (holding ?x))
   :effect (and (on ?x ?y) (clear ?x)
                (handempty) (not (holding ?x))))))

(define (problem bw-1)
  (:objects a b c)
  (:init (ontable a) (ontable b) (ontable c)
         (clear a) (clear b) (clear c) (handempty))
  (:goal (and (on a b) (on b c))))
```

Domain

Predicates & actions —
precondition → effect.

Problem

Objects, initial state & goal.

Plan

Ordered grounded actions.

Plan executing · Blocksworld

Goal reached ✓



goal: (on a b) (on b c)

RESULTING STATE

(on a b) (on b c) (ontable c) (clear a)

Why an Ontology for Planning?

An **ontology** is a formal, shared representation of concepts and their relationships. Modeling planning domains and planner capabilities this way turns scattered metadata into **queryable, reusable knowledge**.



Organize Metadata

Capture domains, planners, heuristics & results in one structured model.



Reason & Query

Answer semantic questions over planning knowledge with SPARQL.



Improve Performance

Select promising planners; extract macros as action-ordering constraints.



Explain & Share

Generate human-readable explanations and promote reuse across the community.

From Gaps to Contributions

LIMITATIONS OF PRIOR WORK

Not reusable / closed

PLANET is comprehensive but not open-sourced — hard to extend or replicate.

Narrow scope

Plan-Taxonomy explains planner functionality but not domains & performance.

Unstructured metadata

IPC performance data is rich but rarely modeled for reasoning.

OUR CONTRIBUTIONS

ONTOLOGY DEVELOPMENT

The most comprehensive open planning ontology to date, driven by competency questions.

USE CASE 1

Identify the most promising planner for a domain (IPC-2011).

USE CASE 2

Generate human-readable explanations for plans via SPARQL.

PART 2

The Planning Ontology

Ontology scope & structure

Competency questions

Use cases: selection & explanation

The Planning Ontology — Structure

19
CLASSES

25
OBJECT PROPERTIES

Domain

What a planner can do

PlanningDomain Action Predicate
Requirement MacroAction

Problem

What must be solved

PlanningProblem InitialState GoalState
ProblemObject

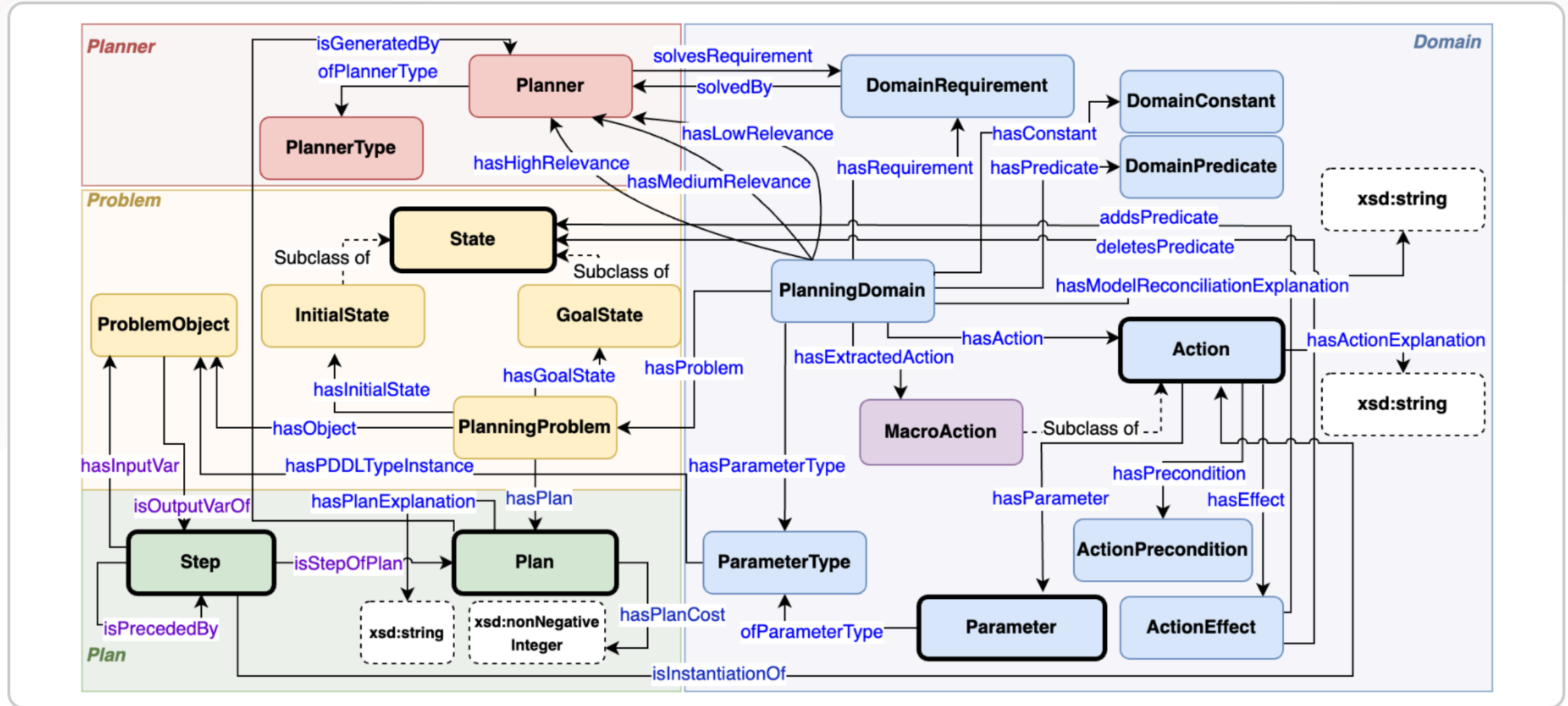
Plan

The solution & its quality

Plan Step PlanCost

Built from IPC data, validated by competency questions, and **extensible beyond IPC / PDDL**. Full schema next →

SINGLE-AGENT PO The Planning Ontology — Full Schema



Classes (Domain · Problem · Plan) and the 25 object properties that connect them.

Competency Questions Drive the Design

Ten competency questions scope what the ontology must answer — each becomes a **SPARQL query template** over the knowledge graph.

■ **planner / plan terms** ■ **domain / problem terms**

C1 What are the different types of **planners** used in automated planning?

C3 What are the available **actions** for a given **domain**?

C5 What are all the **requirements** a given **domain** has?

C7 How many **parameters** does a specific **action** have?

C9 What **requirements** does a given **planner** support?

C2 What is the **relevance** of **planners** in a given problem **domain**?

C4 What **problems** in a **domain** satisfy a given **condition**?

C6 What is the **cost** of generating a **plan** for a given **problem**?

C8 What **planning type** does a specific **planner** belong to?

C10 What are the different **parameter types** present in a **domain**?

From PDDL to a Knowledge Graph



An ontology instance

Domain, problem & plan files become a knowledge-graph instance of the ontology.

Typed nodes & edges

Domains, actions, predicates, objects & plan steps become typed nodes; relations become object properties.

Performance metadata

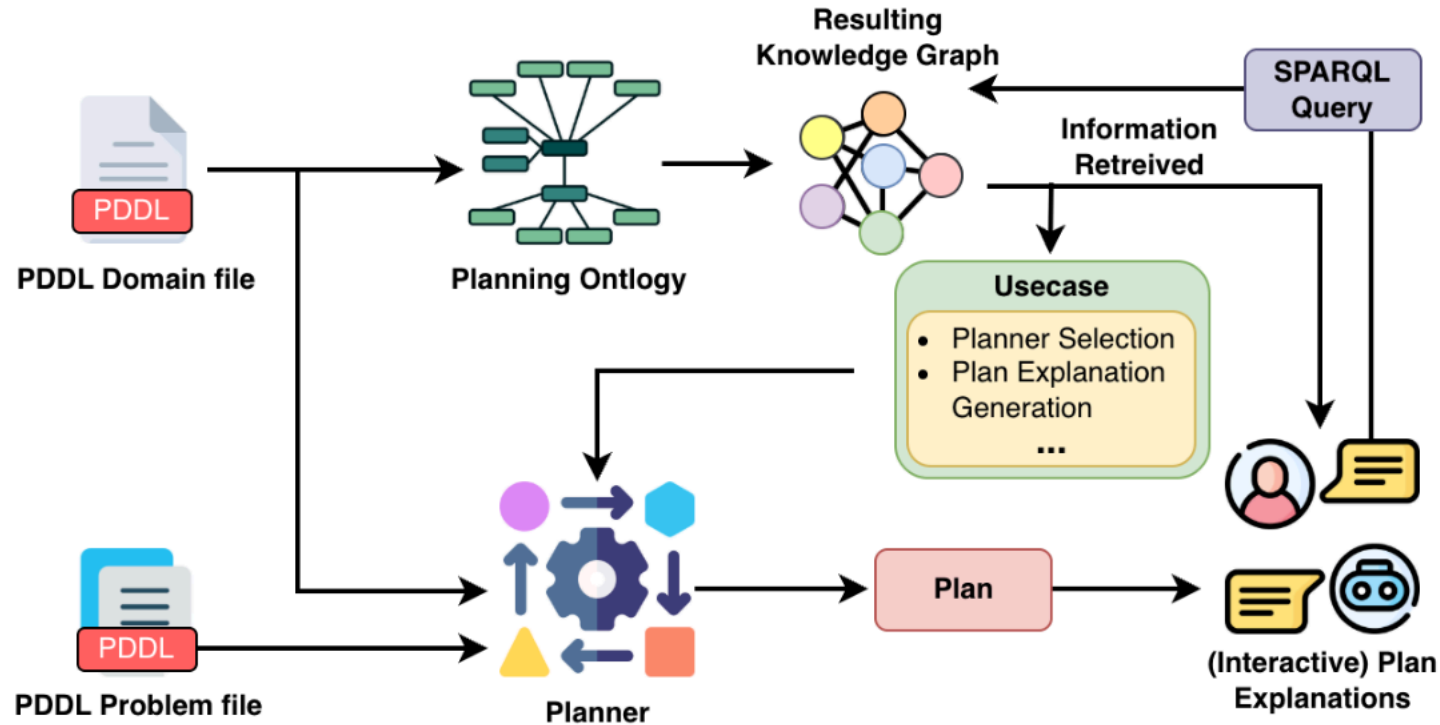
IPC planner results are attached, enabling semantic queries — shown here for Blocksworld.



Knowledge graph for the Blocksworld domain — an instance of the planning ontology.

A Planning Workflow with the Ontology

The **ontology** turns PDDL into a queryable **knowledge graph**; **SPARQL** then powers downstream use cases.



PDDL → Ontology → Knowledge Graph

SPARQL retrieves information

Use cases: planner selection & explanation

Identifying the Most Promising Planner

Challenge

Picking a planner for a new domain is largely trial-and-error across many planner / heuristic combinations.

Ontology-driven approach

1. Encode IPC-2011 performance as a knowledge graph.
2. Query for the best-performing planner per domain.
3. Apply that **ontology policy** instead of a random choice.

POLICY	NODES EXPANDED	SUCCESS RATE
Ontology policy	Fewer	Higher
Random policy	More	Lower

Ontology-guided selection expands fewer nodes and solves more instances than a random policy (IPC-2011).

Bonus: the ontology also yields **macros** — action-ordering constraints that further speed up planning.

Generating Plan Explanations

Challenge

Planners produce correct plans but little human-readable rationale for *what* happens and *why*.

Ontology-driven approach

Run **SPARQL** over the plan knowledge graph, then fill results into natural-language explanation templates.

User survey: ontology-generated explanations rated highly across three explanation types (avg out of 5).

SPARQL · high-level plan summary

```
SELECT ?step ?action ?object WHERE {  
  ?plan :hasStep ?step .  
  ?step :hasAction ?action ;  
        :actsOn    ?object ;  
        :stepNumber ?n .  
} ORDER BY ?n
```

Explanation →

"First unstack B from A, then place A on B and B on C to build the goal tower."

PART 3

The Planning Ontology Tool

PDDL → knowledge graph

Interactive visualization

SPARQL in the browser

An Interactive Tool on Planning.Domains

A **browser plugin** for the Planning.Domains PDDL editor. It turns PDDL into an **RDF/OWL knowledge graph** you can visualize and query — entirely client-side, no server.

Pyodide

Python in the browser

RDFLib

builds the graph

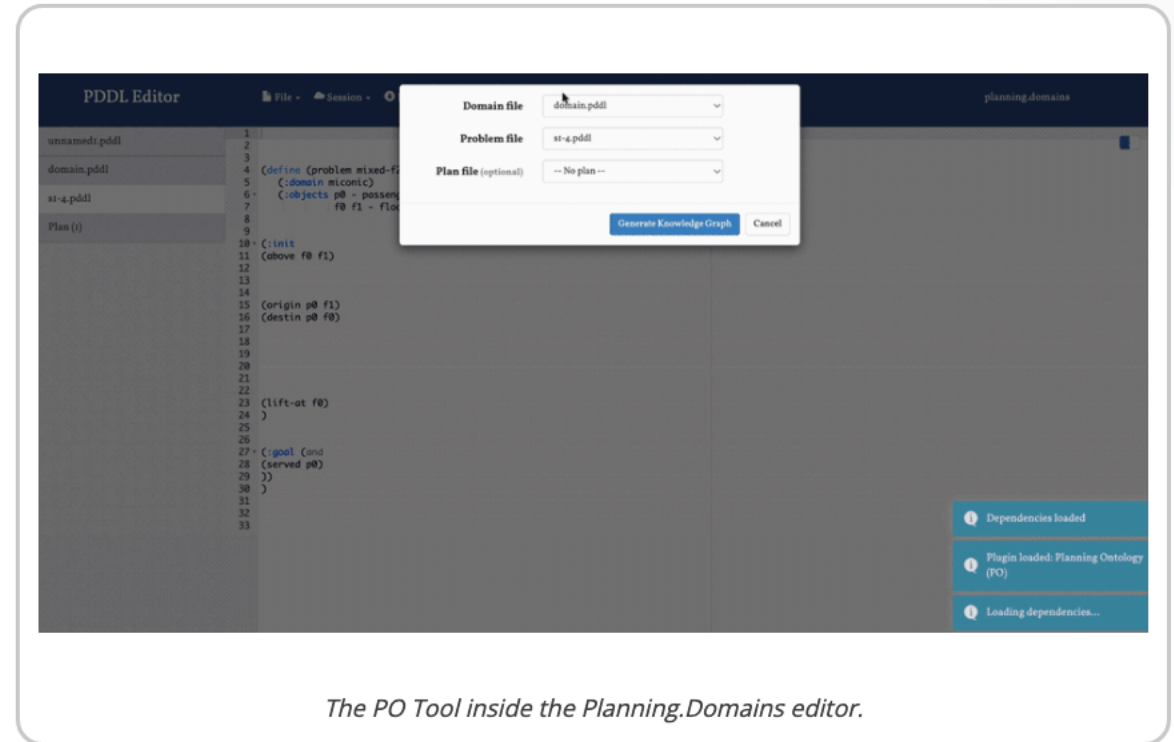
D3.js

interactive visualization

Comunica

SPARQL querying

PURL · github.com/ai4society/planning-ontology-tool-editor.planning.domains



The PO Tool inside the Planning.Domains editor.

PDDL → Knowledge Graph → SPARQL

The screenshot shows the PDDL Editor interface. On the left, a file explorer lists files: unnamed1.pddl, domain.pddl, st-4.pddl, and Plan (1). The main area displays a list of plugins with their descriptions and authors. The 'Planning Ontology Tool' plugin is highlighted. On the right, a panel titled 'planning.domains' shows a plan visualization with a green bar indicating 'Plan is ready' and a green bar indicating 'Task Initiated!'. Below these bars, it says 'Running remote planner...'.

Plugin Name	Description	Author(s)	Install Button
Planning-as-a-Service Plugin	Plugin to call the solver provided by Planning-as-a-Service	Christian Muise, Nir Lipovetzky, Yi Ding	Install
Action Usability Detection	Action usability via deadend detection in AI planning problems.	Christian Muise, Qianyu Zhang	Install
Dovetail	Visualize how predicates change during plan execution using the Dovetail metaphor	Mau Magnaguagno	Install
Heuristic Visualization	Dynamic state-space expansion and visualization of heuristic computation.	Cam Cunningham, Caitlin Aspinall, Ellie Sekine, Christian Muise	Install
Heuristic Viz with Planimation	Plugin modification of Heuristic Visualization to include Planimation generated states.	Nir Lipovetzky, Yi Ding	Install
Misc PDDL Generators	Adds a number of utilities to the editor for generating PDDL.	Christian Muise and Nir Lipovetzky	Install
Planimation	Solver to Animate PDDL Plans.	Nir Lipovetzky (plugin)	Install
Planning Ontology Tool	This plugin transforms PDDL (Planning Domain Definition Language) domain and problem files, and the generated plan files into RDF/OWL knowledge graphs, visualizes them using D3.js, and enables SPARQL querying.	Bharath Muppasani, Bernardo Denkvits, Biplav Srivastava	Install
SDAC Translator	Translates a problem with state-dependent action costs into one without.	Florian Geißer (sdac), Christian Muise (plugin)	Install
Statespace	Visualize state-space explored during planning	Mau Magnaguagno	Install
Timeline Viewer	Visualize the timeline of a plan from a temporal planner's output	Steve Levine	Install
Torchlight	Calls the torchlight software to analyze classical problems.	Jörg Hoffmann (torchlight), Christian Muise (plugin)	Install

Running inside the Planning.Domains editor — entirely in the browser (Pyodide · rdflib · D3.js · Comunica).

IN THE EDITOR

1 · Load PDDL

Pick domain, problem & plan files in Planning.Domains.

2 · Build the graph

Files are parsed and converted to an RDF/OWL knowledge graph.

3 · Visualize

Explore the planning graph interactively with D3.js.

4 · Query

Run SPARQL to answer competency questions over the graph.



PART 4

Multi-Agent: maPO

Explainable MAPF

Schema, conflicts & provenance

The OMEGA dashboard

Why a Multi-Agent Planning Ontology?

Multi-Agent Path Finding (MAPF)

Plan collision-free paths for many agents on a shared graph — core to robotics & logistics.

Powerful but opaque

Solvers (CBS / ICBS, PIBT, RL / PRIMAL, MAPF-GPT) expose little about *why* decisions are made.

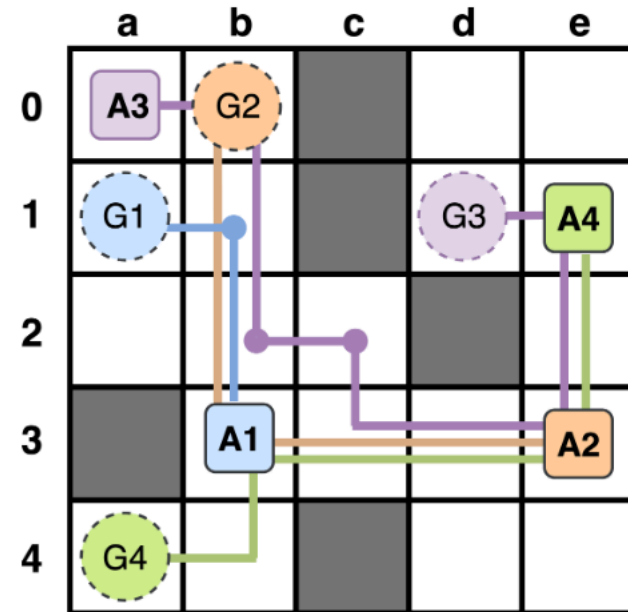
Explanations are required

Safety-critical & regulatory settings demand transparent, human-readable answers.

Prior work is fragmented

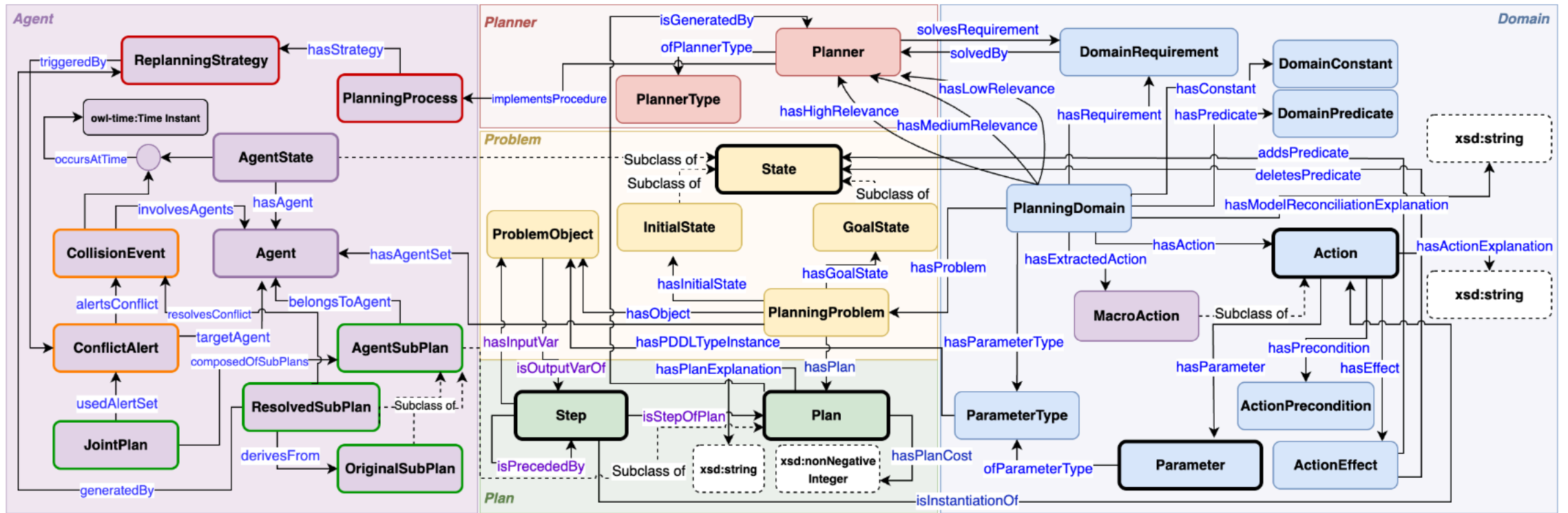
Visual segmentation, ASP why / why-not, taxonomies — with no unified, queryable model.

maPO turns raw MAPF traces into a queryable knowledge graph for on-demand explanations — **planner-agnostic**.



A MAPF instance: agents (squares) must reach their goals (circles) without collisions.

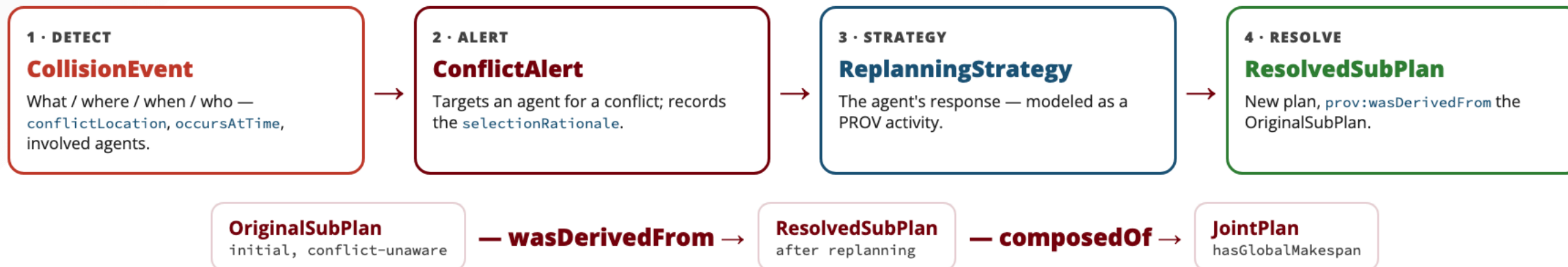
MULTI-AGENT · MAPO The maPO Schema — Full View



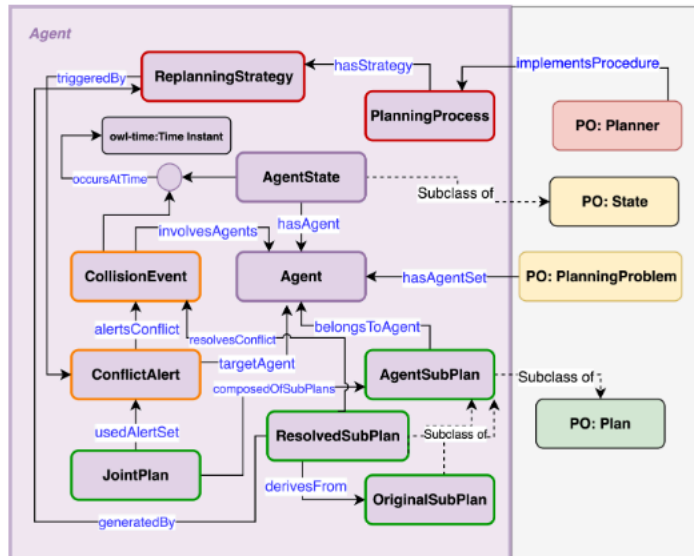
Agent, AgentState, sub-plans, CollisionEvent, ConflictAlert, ReplanningStrategy & JointPlan — extending PO.

Modeling the Conflict-Resolution Lifecycle

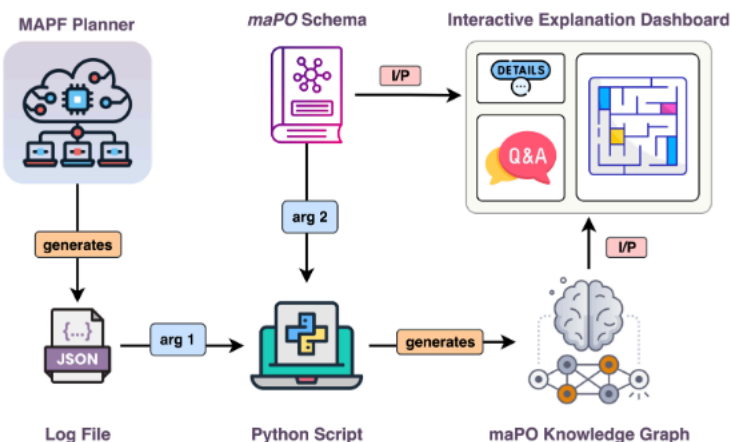
maPO formalizes the full causal chain from **conflict detection** to **resolution** — every step grounded in **PROV-O** for traceability.



From Planner Logs to Explanations



(a) Multi-Agent Planning Ontology (maPO) Schema



(b) System Architecture

Planner-specific input

Raw MAPF traces & optional args — not a custom explanation API.

Ontology mapping

generate_kg.py asserts RDF triples automatically.

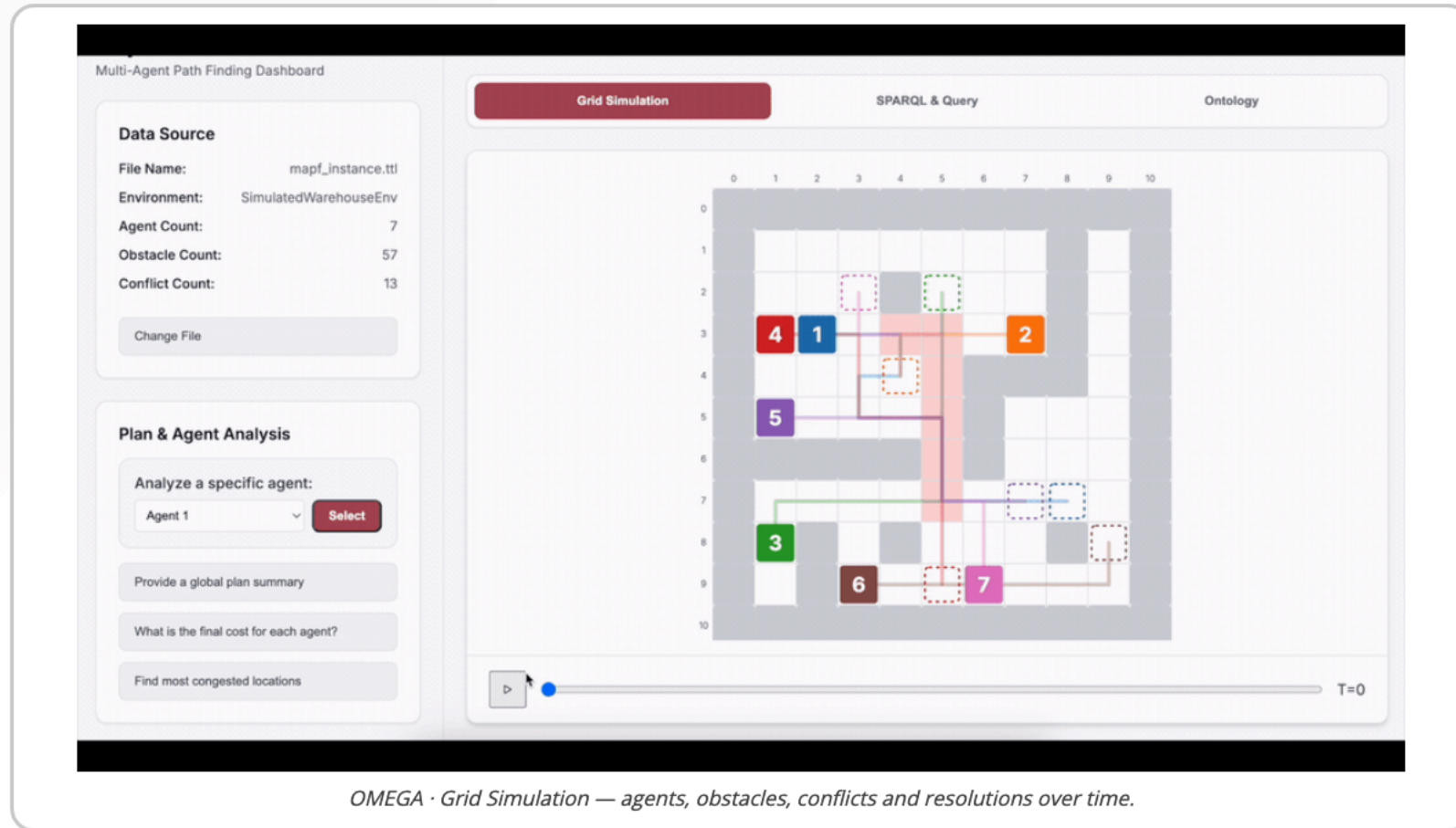
Planner-agnostic output

One graph drives queries, replay & explanations.

```
python generate_kg.py --log_file
<json> --output <ttl>
```

System architecture — planner-specific logs are mapped into a planner-agnostic, SPARQL-queryable knowledge graph.

Replay the Plan on the Grid



Grid Simulation

SPARQL & Query

Ontology

STEP THE TIMELINE

Watch the plan unfold

Scrub the timeline to see each agent move, collide and re-route on the grid.

Color-coded agents & goals

Squares are agents, circles are goals; conflict cells are highlighted.

Grounded in the graph

Every position & conflict shown here is a node in the maPO knowledge graph.

Ask Questions, Get Explanations

Change File

Grid Simulation

SPARQL & Query

Ontology

Analysis for Agent 3

← Go Back

What is Agent 3's final plan?

The final plan for **Agent 3** has a cost of **16.0**.

Summary: The agent starts at (8,1), then **moves** to (7,1), then **moves** to (7,2), then **moves** to (7,3), then **moves** to (7,4), then **moves** to (7,5), then **waits** at (7,5) for 2 step(s), then **moves** to (7,4), then **waits** at (7,4) for 2 step(s), then **moves** to (7,5), then **moves** to (6,5), then **moves** to (5,5), then **moves** to (4,5), then **moves** to (3,5), then **moves** to (2,5).

Full Path:

(8,1) → (7,1) → (7,2) → (7,3) → (7,4) → (7,5) → (7,5) → (7,5) → (7,4) → (7,4) → (7,4) → (7,5) → (6,5) → (5,5) → (4,5) → (3,5) → (2,5)

[View in SPARQL Tab](#)

Grid Simulation

SPARQL & Query

Ontology

▶

T=0

OMEGA · Plan & Agent Analysis — natural-language explanations generated from SPARQL over the graph.

Grid Simulation

Plan & Agent Analysis

Ontology

Q · What is Agent 3's final plan?

"The agent starts at (8,5), moves to (7,0), waits at (7,5), then reaches its goal — cost 16.0."

Per-agent questions

Plan summary, conflicts, why an agent waited or re-routed, congested cells.

Provenance-grounded

Answers trace the resolution history via PROV-O — auditable, not hand-written.

Browse the Knowledge Graph

Explainable MAPF
Multi-Agent Path Finding Dashboard

Data Source

File Name: mapf_instance.ttl
Environment: SimulatedWarehouseEnv
Agent Count: 7
Obstacle Count: 57
Conflict Count: 13

Change File

Plan & Agent Analysis

Analyze a specific agent:
Agent 1 Select

Provide a global plan summary

What is the final cost for each agent?

Find most congested locations

Grid Simulation **SPARQL & Query** **Ontology**

WebVOWL 1.1.7

Collapsing filter activated. The number of visualized elements has been automatically reduced. Use the degree of collapsing slider in the filter menu to adjust the visualization.

Note: A performance decrease could be experienced with a growing amount of visual elements in the graph.

Got It

Multi-Agent Planning Ontology

▼ Description

An ontology for representing knowledge in Multi-Agent Path Finding (MAPF) problems. This ontology defines concepts such as agents, plans, conflicts, and replanning strategies to support explainable AI in MAPF.

► Metadata

► Statistics

► Selection Details

Search Export Filter Options Modes Reset Pause About

OMEGA · Ontology — the live maPO knowledge graph, explored interactively with WebVOWL.

Grid Simulation

Plan & Agent Analysis

Ontology

See the schema live

Classes & object properties of maPO, laid out as an interactive graph.

Drag, filter, zoom

Explore how agents, conflicts & sub-plans connect — the same graph the queries run over.

Agent

CollisionEvent

JointPlan

PROV-O

Evaluating maPO Explanations

Two complementary evaluations — automatic **readability** (cognitive load) and a human **user study** across RL, CBS & ICBS scenarios.

94.4

FLESCH READING EASE

“very easy” · grade ~1–5

95.2%

PREFERRED MAPO

159 / 167 judgements

4.40

CLARITY / 5

$p < .001$

N = 28

PARTICIPANTS

graduate & undergraduate

Cognitive-load metrics (FRE · ARI · CLI) confirm the explanations are **linguistically simple** and easy to comprehend — even for complex 7-agent scenarios.

** maPO & OMEGA work additionally with **Ritirupa Dey** (University of South Carolina).*

CONCLUSION

What We Built



PLANNING ONTOLOGY

An open, structured representation of planning knowledge — validated by competency questions and demonstrated on planner selection & explanation.



THE PO TOOL

A browser plugin that turns PDDL into a knowledge graph with D3 visualization and SPARQL querying, live on Planning.Domains.



MAPO

A multi-agent extension for explainable MAPF — provenance-aware, planner-agnostic, with a strong user-study result.

Structuring planning knowledge as an ontology makes planners **more performant** and their behaviour **explainable** — from single-agent PDDL to multi-agent path finding.

Where This Goes Next



Language Models & Ontologies

- › Ground LLM plan explanations in the ontology for context-aware, faithful narratives.
- › Natural-language SPARQL querying over planning knowledge graphs.
- › Template learning to adapt explanations to the user.



Neuro-Symbolic Representations

- › Integrate symbolic ontology knowledge with neural planners & learners.
- › Use structured constraints (e.g., macros) to guide and explain learned policies.
- › Safety-rule checking & conflict-driven diagnostics.

Also: extend **maPO** to dynamic, real-world multi-agent & robotics settings (via SOSA/SSN), richer conflict taxonomies, and temporal / numeric planning.

ICAPS 2026 TUTORIAL

Thank You!

Explore the **Planning Ontology**, the tool, and **maPO** — scan to dive in.



PO Website

ai4society.github.io/planning-ontology



Ontology PURL

purl.org/ai4s/ontology/planning



PO Tool

github.com/ai4society/planning-ontology-tool



maPO

ai4society.github.io/ma-planning-ontology

Contact: Bharath Muppasani · Nitin Gupta · Biplav Srivastava · Vignesh Narayanan — AI Institute of South Carolina (AIISC), University of South Carolina